

CSCI5409

Advanced Topics in Cloud Computing

CloudNotes

A 3-Tier Cloud Note-Taking Application

Mid Term Project Report

Student Name:- Hema Sen

Student ID:- B01050989

Course:- CSCI5409 – Winter 2026

Instructor:- Prof. Lu Yang

Submission Date:- 8TH of March 2026

1. Project Introduction

1.1 What is CloudNotes ?

CloudNotes is a fully cloud-native, three-tier note-taking and file management web application deployed on Amazon Web Service (AWS). The application allows users to create, read, update, and delete (CRUD) personal notes stored in a managed relational database, as well as upload, download, and delete files stored in scalable cloud object storage.

The application was built to demonstrated the end-to-end design, deployment, and operation of a production-style 3-tier web application entirely on AWS, provisioned fully through Infrastructure as Code (IaC) using Terraform.

1.2 Purpose and Use Case

CloudNotes addresses a common real-world need: a lightweight, always-available, web-based personal productivity tool. Users can:

Take and store notes with title and body content, accessible from any browser.
Upload and retrieve files (document, images, PDFs) from secure cloud storage.
Perform all data operation (create, read, update, delete) through an intuitive UI
Access a persistent data store that survive instance restarts or failures.

1.3 Target Users

The primary users are students and knowledge workers who need a simple, reliable note-taking tool accessible from any device with a browser. The application does not require any local software installation and is accessible via a public IP address immediately after provisioning.

1.4 Performance Target

Page Load Time	Under 2 second for the main notes review
File Upload Size	Up to 16 MB per file (Flask default limit)
Database Response	Under 100 ms for standard SELECT/INSERT/UPDATE queries
Availability Target	99.9% uptime (single-AZ, suitable for lab/demo use)
Concurrent Users	10-50 simultaneous users on t2.micro

2. AWS Service Category Requirements

2.1 Service Selected

Category	Service Selected	Requirement Met
Compute	Amazon EC2 (t2.micro)	Yes – at least one
Storage	Amazon S3	Yes – at least one
Networking & Content Delivery	Amazon VPC (Default VPC + Security Groups)	Yes – at least one
Database	Amazon RDS for MySQL 8.0 (db t3.micro)	Yes – at least one

2.2 Compute: Amazon EC2 vs. Alternatives

Selected: Amazon EC2 (t2.micro)

Amazon EC2 was chosen as the compute layer because it provides full control over the operating system, runtime environment, and application lifecycle. The Flash web server runs directly on the EC2 instance using Python 3, giving complete flexibility in how dependencies are installed and the application is started.

Alternative	Why Not Chosen
AWS Lambda	Serverless functions are stateless and optimized for event-driven workloads. A Flask application with persistent DB connections is better suited to a long-running server.
AWS Elastic Beanstalk	Abstract away too much infrastructure control; harder to customize the exact startup sequence needed to wait for RDS initialization.
AWS App Runner	Designed for container apps; would require Dockerizing the application, adding complexity not needed for this project scope.
Amazon Lightsail	Simple to use but fewer configuration options and not well-suited for integration with RDS and IAM instances profile.

2.3 Storage: Amazon S3 vs. Alternatives

Selected: Amazon S3

Amazon S3 was selected for file storage because it provides virtually unlimited, highly durable (99.99999999%) object storage with a simple API. The boto3 SDK seamlessly with

Flask and the EC2 instances accesses S3 using the attached LabRole. IAM instance profile, eliminating the need to manage credentials.

2.4 Networking: Amazon VPC vs. Alternatives

Selected: Amazon VPC (Default VPC + Security Groups)

The AWS Academy Learner Lab restricts the ability to create new VPC resources (CreateVpc is explicit denied by the vpc-cancel-cred IAM policy). Therefore, the existing Default VPC was used, which provides subnets across multiple Availability Zones in us-east-1. Security Groups were created within the Default VPC to enforce tier isolation between the EC2 application layer and the RDS database layer.

Alternatives	Why Not Chosen
Custom VPC with Private Subnets	Ideal for production isolates RDS in private subnets with no internet exposure. Not feasible in this lab environment due to CreateVpc permission restrictions.
Amazon CloudFront	CDN for static asset delivery and DDoS PROTECTION. Valuable for production, but adds complexity and cost beyond the scope of this academic project.
Elastic Load Balancing	Required for multi-instances deployments. CloudNotes currently runs on a single EC2 instances; ELB would be the next step for high availability.
Amazon API Gateway	Best suited for REST API backends (Lambda or containerized). A Flask monolith served from EC2 does not require API Gateway.

2.5 Database: Amazon RDS vs. Alternatives

Selected: Amazon RDS for MySQL 8.0

Amazon RDS was selected because it provides a fully managed relational database that handles backups, patching, and replication automatically. MySQL was chosen as the database engine because it is widely used, well-supported by the PyMySQL Python library, and familiar to most developers.

Alternative	Why Not Chosen
Amazon DynamoDB	A NoSQL key-value store. CloudNotes stores structured relational data (notes with id, title, content, timestamps). A relational data is the more natural fit.

Amazon ElasticCache (Redis)	In memory cache, not a persistent data store. Would complement RDS as a caching layer but cannot replace it.
Amazon Aurora	AWS-native MySQL/PostgreSQL compatible engine with better performance and availability. More expensive and slightly more complex to configure; RDS MySQL is sufficient for this scale.
SQLite on EC2	No managed service benefits. Data is stored on ephemeral EC2 instance storage and would be lost when the instance terminates.

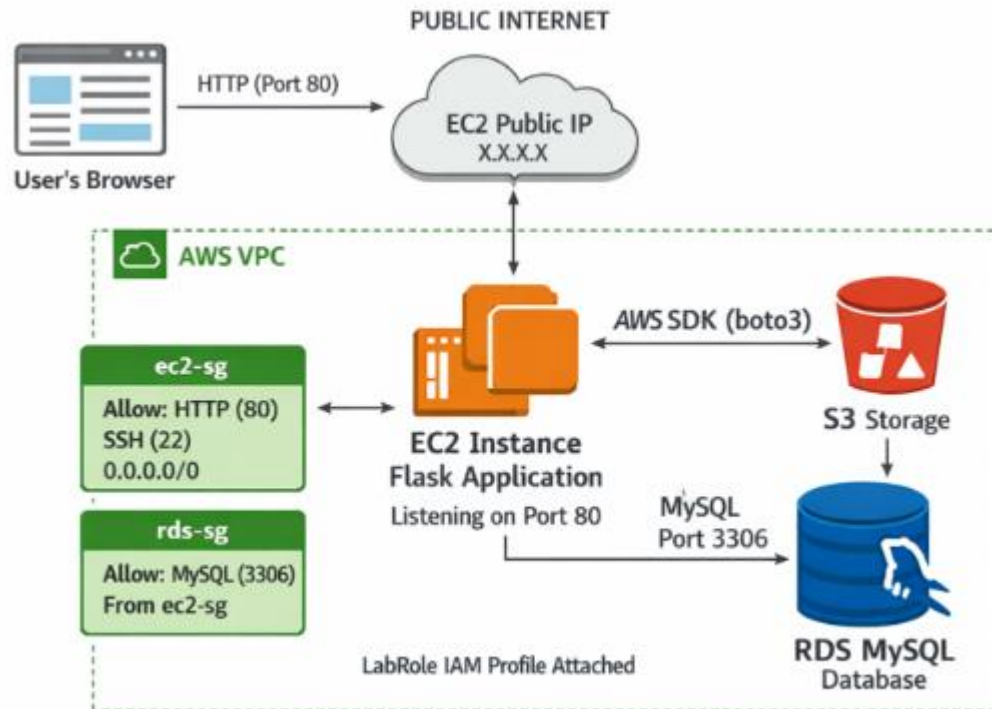
3. Final Architecture

3.1 Architecture Overview

CloudNotes follows a classic 3-tier web application architecture deployed entirely on AWS. The three tiers are clearly separated both logically and at the network security level using Security Groups.

Tier	Components	AWS Service
Presentation / Application Tier	Flask Web Server	Amazon EC2 t2.micro
Storage Tier	File Object Store	Amazon S3 Bucket
Data Tier	Relational Database	Amazon RDS MySQL 8.0 (db.t3.micro)
Networking	Virtual Private Cloud	Default AWS VPC + Security Groups
IaC	Infrastructure Provisioning	HashiCorp Terraform

3.2 Architecture Diagram



3.3 How Components Integrate

When a user visits the CloudNotes URL in their browser:

1. The browser sends an HTTP GET request to `http://<EC2-Public-IP>`.
2. Flask (running on port 80) handles the request, connects to RDS MySQL, queries all notes, and call S3 to list stored files.
3. Flask renders an HTML response using Jinja2 templates and returns the fully assembled page to the browser.
4. For POST operation (add/edit/delete note, upload/delete file), Flask processes the form data, executes SQL or S3 API calls, and redirects back to the homepage with a success toast message.

3.4 Data Details

Aspect	Detail
Data Format – Notes	Structured relational data: id (INT), title (VARCHAR 255), content (TEXT), created_at (TIMESTAMP), updated_at (TIMESTAMP)
Data Format – Files	Unstructured binary objects stored as-is in S3; metadata (Key, Size, LastModified) retrieved via list_objects_v2 API
Data Source	User-generated content entered through the CloudNotes web UI
Data Size	Notes: typically < 10 KB each; Files: up to 16 MB per object (Flask default)
Notes Storage	Amazon RDS MySQL 8.0 on db.t3.micro with 20 GB gp2 EBS volume
File Storage	Amazon S3 bucket (cloudnotes-storage-hema-4774636) in us-east-1
Data Persistence	RDS data persists independently of EC2. S3 data persists indefinitely. Both survive EC2 reboots/replacements.

3.5 Programming Languages and Code Components

Language / Tool	Where Used	Why
Python 3 (Flask)	Web application server – routing, templating, DB queries, S3 API calls	Lightweight, rapid development, excellent AWS SDK support via boto3
PyMySQL	MySQL database connector for Python	Pure Python, no system dependencies, works on Amazon Linux 2

Language / Tool	Where Used	Why
boto3 (AWS SDK)	S3 upload, download, list, delete operations	Official AWS SDK for Python, uses IAM instance profile automatically
Jinja2	HTML templating within Flask responses	Bundled with Flask; enables dynamic HTML generation server-side
HCL (Terraform)	Infrastructure as Code – all AWS resource provisioning	Declarative, provider-agnostic IaC; single terraform apply deploys everything
Bash	EC2 user_data startup script – installs Python, pip, runs Flask	Native to Amazon Linux 2; used for bootstrap automation
HTML / CSS / JS	Frontend UI – forms, modals, tab switching, toast notifications	No framework needed; pure HTML/CSS/JS keeps dependencies minimal

3.6 Deployment Process

The entire infrastructure and application are provisioned in a single automated process using Terraform. There is zero manual intervention required after running the commands below:

```
PS C:\Users\Hema Sen\Desktop\Cloud-Computing-Project-1> terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Hema Sen\Desktop\Cloud-Computing-Project-1>
```



```

PS C:\Users\Hema Sen\Desktop\Cloud-Computing-Project-1> terraform plan
data.aws_vpc.default: Reading...
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-05024c2628f651b80]
data.aws_vpc.default: Read complete after 1s [id=vpc-0455bb55bf4c4e3d4]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=us-east-1]
data.aws_subnet.sub2: Reading...
data.aws_subnet.sub1: Reading...
data.aws_subnet.sub2: Read complete after 0s [id=subnet-0df8a534265221c19]
data.aws_subnet.sub1: Read complete after 0s [id=subnet-0292231ecf47c99a4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql will be created
+ resource "aws_db_instance" "mysql" {
  + address                = (known after apply)
  + allocated_storage      = 20
  + apply_immediately      = false
  + arn                    = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone       = (known after apply)
  + backup_retention_period = (known after apply)

```

```

PS C:\Users\Hema Sen\Desktop\Cloud-Computing-Project-1> terraform apply
data.aws_ami.amazon_linux: Reading...
data.aws_vpc.default: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-05024c2628f651b80]
data.aws_vpc.default: Read complete after 1s [id=vpc-0455bb55bf4c4e3d4]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=us-east-1]
data.aws_subnet.sub1: Reading...
data.aws_subnet.sub2: Reading...
data.aws_subnet.sub1: Read complete after 0s [id=subnet-0292231ecf47c99a4]
data.aws_subnet.sub2: Read complete after 0s [id=subnet-0df8a534265221c19]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.mysql will be created
+ resource "aws_db_instance" "mysql" {
  + address                = (known after apply)
  + allocated_storage      = 20
  + apply_immediately      = false
  + arn                    = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone       = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target           = (known after apply)
  + backup_window          = (known after apply)

```

Terraform provisions resources in this order:

- Security Groups (ec2-sg, rds-sg) are created in the Default VPC
- S3 bucket is created with a fixed name
- RDS DB Subnet Group is created using the two default subnets
- RDS MySQL instance is provisioned (takes approximately 5–7 minutes)
- EC2 instance is launched with depends_on = [aws_db_instance.mysql], ensuring RDS is ready first
- EC2 user_data script runs: installs Python/pip/Flask, writes app.py, starts Flask as root on port 80
- The init_db() function retries the DB connection up to 10 times (30-second intervals) to handle any remaining RDS warm-up time
- terraform output prints the app URL: <http://<EC2-Public-IP>>

4. Security Analysis

4.1 Security Measures Implemented

Security Mechanism	Implementation	Details
Network Tier Isolation	Security Groups	RDS Security Group (rds-sg) only allows port 3306 from ec2-sg. The database is never directly accessible from the internet.
RDS Not Publicly Accessible	publicly_accessible = false	RDS has no public IP or DNS endpoint; it is only reachable within the VPC.
IAM Instance Profile	LabInstanceProfile on EC2	EC2 uses the LabRole IAM instance profile to access S3 without any hardcoded AWS credentials in code.
S3 Access via SDK	boto3 + IAM role	Files in S3 are not publicly accessible. Users can only access files through the Flask download endpoint, which streams the file from S3.
No Direct DB Internet Exposure	VPC-internal only	MySQL port 3306 is blocked from 0.0.0.0/0; it is only reachable from the EC2 instance.

4.2 Known Vulnerabilities and Mitigations

Vulnerability 1: Hardcoded Database Credentials

The RDS username and password (admin / password123) are hardcoded in both the Terraform file and the Python application. This is a significant security risk in production.

Mitigation: Use AWS Secrets Manager to store credentials. The EC2 instance would retrieve the secret at startup via boto3, and the secret would never appear in source code or IaC files.

Vulnerability 2: HTTP (No TLS/HTTPS)

The application runs on HTTP port 80. All traffic including note content and file uploads is transmitted in plaintext.

Mitigation: Deploy an Application Load Balancer (ALB) with an SSL/TLS certificate from AWS Certificate Manager (ACM). The ALB would terminate HTTPS and forward HTTP to the EC2 backend.

Vulnerability 3: SSH Open to 0.0.0.0/0

Port 22 is open to all IP addresses for debugging convenience. This exposes the instance to brute-force SSH attacks.

Mitigation: Restrict the SSH ingress rule to a specific trusted IP CIDR (e.g., your office or home IP). In production, use AWS Systems Manager Session Manager to eliminate the need for open SSH entirely.

Vulnerability 4: No Input Sanitization / SQL Injection Prevention

The Flask application uses parameterized queries (%s placeholders with PyMySQL), which does protect against basic SQL injection. However, there is no input validation on note content length, file type, or file size.

Mitigation: Add server-side validation for input length limits, allowed file MIME types, and maximum file sizes. Use AWS WAF on an ALB to block common web attack patterns (SQL injection, XSS).

Vulnerability 5: No Authentication

Anyone with the EC2 public IP can access, modify, and delete all notes and files. There is no user authentication or authorization.

Mitigation: Integrate Amazon Cognito for user authentication. Each note and file would be associated with a user ID, and Flask would validate JWT tokens from Cognito before processing requests.

4.3 IAM Considerations

Due to AWS Academy Learner Lab restrictions, full IAM configuration is not possible. In a production deployment, the following IAM improvements would be implemented:

- Principle of Least Privilege: The EC2 IAM role would have an inline policy granting `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`, `s3:ListBucket` only for the specific CloudNotes S3 bucket ARN.
- RDS IAM Authentication: Replace password authentication with IAM database authentication, eliminating static database passwords entirely.
- Resource-based S3 Bucket Policy: Deny all public access to the S3 bucket and only allow access from the specific EC2 IAM role ARN.

- VPC Endpoints for S3: Use an S3 Gateway VPC Endpoint so EC2-to-S3 traffic never traverses the public internet.

5. Cost Analysis

5.1 Monthly Cost Breakdown (AWS Pricing – us-east-1)

Service	Configuration	Estimated Monthly Cost
Amazon EC2	t2.micro – On-Demand (730 hrs/month)	~\$9.50 USD
Amazon RDS MySQL	db.t3.micro – Single-AZ, 20 GB gp2 storage	~\$27.00 USD
Amazon S3	First 50 GB storage + 10,000 PUT requests + 10,000 GET requests	~\$1.50 USD
Data Transfer	Up to 1 GB outbound per month (free tier: 100 GB/month free)	~\$0.00 USD
Security Groups / VPC	Default VPC – no additional charge	\$0.00 USD
Total Estimated		~\$38.00 USD / month

5.2 Up-Front Costs

CloudNotes uses exclusively On-Demand pricing with no Reserved Instance commitments. There are no upfront costs. The application can be fully destroyed and re-provisioned at any time with zero sunk cost.

5.3 Additional Costs (Production Readiness)

Addition	Estimated Cost
Application Load Balancer (for HTTPS)	~\$18.00/month base + LCU charges
AWS Certificate Manager (ACM) SSL Cert	\$0.00 (free for ALB-attached certs)
Amazon Cognito (first 50,000 MAU)	\$0.00 (free tier)
AWS Secrets Manager (1 secret)	~\$0.40/month

Addition	Estimated Cost
Amazon CloudWatch (basic monitoring)	\$0.00 (free tier covers basic EC2/RDS metrics)

5.4 Cost Optimization Alternatives

Option 1: Use AWS Free Tier (Lowest Cost)

For the first 12 months of a new AWS account, t2.micro EC2 and db.t3.micro RDS are both included in the Free Tier (750 hours/month each). Total cost in the first year: ~\$0 for compute and database.

Option 2: Replace RDS with DynamoDB (Serverless NoSQL)

Amazon DynamoDB offers a free tier of 25 GB storage and 25 provisioned capacity units permanently. Migrating from MySQL to DynamoDB would eliminate the ~\$27/month RDS cost, reducing total costs to under \$5/month. The tradeoff is the need to redesign the data model from relational to key-value/document format.

Option 3: Replace EC2 with AWS Lambda + API Gateway (Serverless)

Refactoring the Flask application as Lambda functions behind API Gateway would reduce compute costs to near-zero for low-traffic usage (first 1 million requests/month are free). However, this requires a significant architectural change and introduces cold-start latency.

Option 4: Use Amazon Lightsail

Amazon Lightsail bundles compute, storage, and networking into predictable monthly prices starting at \$3.50/month for a small instance. For simple web apps, Lightsail significantly reduces cost and operational complexity, though it offers less fine-grained control than EC2 + RDS.

6. Future Development

6.1 Next Features

Feature	AWS Service / Approach	Priority
User Authentication	Amazon Cognito User Pools + JWT token validation in Flask	High
HTTPS / TLS Encryption	Application Load Balancer + AWS Certificate Manager (ACM)	High
Note Sharing Between Users	RDS schema addition (shared_with column) + Cognito user IDs	Medium
Full-Text Note Search	Amazon OpenSearch Service (Elasticsearch-compatible)	Medium
Automatic Backups	AWS Backup + RDS automated snapshots	Medium
Email Notifications	Amazon SES – notify users of shared notes or storage limits	Low
File Preview in Browser	Amazon S3 Pre-Signed URLs + CloudFront CDN	Low
Real-Time Collaboration	AWS AppSync (GraphQL subscriptions) + DynamoDB Streams	Low

6.2 Scalability Evolution

Phase 1 – High Availability (Current Bottleneck)

The current architecture has a single point of failure: the EC2 instance. If the instance fails, the application is unavailable. The immediate next step is:

- Deploy an Application Load Balancer (ALB) in front of EC2
- Create an EC2 Auto Scaling Group with a minimum of 2 instances across 2 Availability Zones
- Switch RDS to Multi-AZ deployment for automatic database failover

Phase 2 – Containerization

Containerize the Flask application using Docker and deploy to Amazon ECS (Elastic Container Service) with Fargate. This eliminates the need to manage EC2 instance lifecycle and enables faster scaling with zero server management.

Phase 3 – CI/CD Pipeline

Implement a fully automated deployment pipeline using:

- AWS CodeCommit – source code repository
- AWS CodeBuild – automated testing and Docker image building
- AWS CodePipeline – orchestrates build, test, and deployment stages
- Amazon ECR – stores Docker container images

Phase 4 – Observability

Add comprehensive monitoring and alerting:

- Amazon CloudWatch – custom metrics, dashboards, and alarms for EC2 CPU, RDS connections, and S3 request rates
- AWS X-Ray – distributed tracing to identify performance bottlenecks in Flask request handlers
- Amazon CloudWatch Logs – centralized log aggregation from all EC2 instances

7. Conclusion

CloudNotes successfully demonstrates the design and deployment of a complete, production-style 3-tier web application on AWS using Infrastructure as Code. The application meets all four AWS service category requirements (Compute, Storage, Networking, Database), is fully provisioned by a single terraform apply command with no manual intervention, and delivers meaningful real-world functionality through a polished CRUD user interface.

The architecture reflects careful consideration of AWS service tradeoffs, security best practices applicable within the AWS Academy lab constraints, and a realistic cost analysis grounded in current AWS pricing. The identified vulnerabilities (hardcoded credentials, no HTTPS, no authentication) are acknowledged with concrete mitigation strategies that would be implemented in a production deployment.

The future development roadmap outlines a clear path from the current single-instance prototype to a fully scalable, highly available, containerized, and observable cloud-native

application — demonstrating a thorough understanding of the AWS ecosystem and modern cloud computing principles.